TECHNICAL REFERENCE

GhostBox Protocol

The Sovereign Communication & Permissions Specification: identity, transport, discovery, and non-bribable vault stewardship, with the corrected reference implementation.

IMPLEMENTATION REFERENCE

Version **0.1.0** · Working Draft

May 2026 · AGPL-3.0-or-later

Cory A. Ottenwess

Companion text:
Notes from an Acceleration Native, App. B & Ch. 11

Contents

1. About This Reference

This document is the implementation reference for two interlocking pieces of sovereignty infrastructure. The **GhostBox Protocol** is a zero-identity, asynchronous, dead-drop communication protocol: it lets two parties exchange messages and assets without the transport ever learning who is talking to whom. The **Companion Permissions Layer** is the access-control model that sits between a user's personal data vault and the outside world, mediated by an AI companion that acts as a non-bribable gatekeeper.

They are specified together because they share one design decision applied at two layers: remove the intermediary's ability to extract value from a user's data by removing the intermediary's access to that data. GhostBox applies it to message transport; the Permissions Layer applies it to vault access.

1.1 What this is not

This document does not specify the vault's storage format, the companion's reasoning model, or the Model Context Protocol itself. Those are separate prerequisites. This document specifies the boundary: how the world reaches the vault, and how the vault reaches the world.

1.2 Requirement language

This reference uses RFC 2119 / RFC 8174 keywords. `MUST`, `MUST NOT`, `REQUIRED`, and `SHALL` are absolute. `SHOULD` and `RECOMMENDED` indicate that valid reasons to deviate may exist but the full implications must be understood. `MAY` and `OPTIONAL` are discretionary. A conformant implementation MUST satisfy every MUST in sections 2 through 6. Section 7 is REQUIRED for a sovereign companion deployment and OPTIONAL for a transport-only implementation.

1.3 Divergences from the published book

The book's Appendix B contains illustrative reference code. This reference corrects it for production use and flags the differences so they read as intentional.

> AUTHORITATIVE
>
> Where this document and the book disagree, **this document governs for implementers**. The book remains accurate as illustration. The two corrected items are salt derivation (§3.3) and the Argon2id parameter ladder (§3.4). The reference implementation additionally strengthens entropy enforcement (§3.5) beyond the book's sketch.

2. Architecture: The Spectre Stack

GhostBox is organized as three independent layers. Independence is a hard requirement: **compromise of one layer MUST NOT compromise another**. Each layer is progressively more tangible, from the invisible and private, through the permissioned and accessible, to the voluntary and present.

Corporeal Layer DISCOVERY §5

Voluntary presence. Public Mask · Lobby · Ghost Channel · four discovery modes · five-state relationship model.

Specter Layer TRANSPORT §4

Passive dead-drop. PUT to Locator Hash · challenge-response GET · TTL purge. The server that cannot link.

Spirit Layer IDENTITY §3

Invisible. Exists only in user memory and transient RAM. Quad-Key → Argon2id → Locator Hash (public) + Access Token (private).

The Companion Permissions Layer (§7) is not part of the Spectre Stack; it is the consumer of it. The companion uses the Corporeal Layer's relationship model as the vocabulary for vault access control, and uses the Specter Layer as its communication transport. The personal data vault sits behind the companion and is out of scope for this document.

3. Spirit Layer — Identity

The Spirit Layer is invisible. It exists only in the user's memory and, transiently, in device RAM during an active session. No server holds it. No registration touches it. It cannot be seized, because it has no physical location, and it cannot be revoked, because it has no third-party administrator. Identity is mathematical, not administrative: a user generates a **Quad-Key**, and a memory-hard derivation function transforms it into a public **Locator Hash** and a private **Access Token**.

3.1 The Quad-Key

A Quad-Key is a composite of **four or more** high-entropy factors chosen by the user. The derivation is agnostic to factor type and requires only sufficient total entropy. Factors MAY be drawn from any combination of four classes.

Class	Examples	Notes
know something you know	passphrases, word sets, codes	Most portable; the original model.
are something you are	voiceprint, fingerprint, iris, typing-cadence *hashes*	One factor among several only. Biometrics cannot be rotated if leaked.
have something you have	hardware token, SSI credential, hash of an attested document's properties	The document never enters the system; only a hash of attested properties does.
who someone who confirms you	designated social-recovery contacts	A protocol the user controls, not a platform reset.

Normative requirements:

- An implementation MUST accept a variable composite; different users will use different factor combinations.
- The composite MUST reach a minimum entropy floor of **128 bits** before derivation proceeds. Implementations MUST estimate and enforce this at key-creation time and MUST refuse to derive below it (§3.5).
- The factor composition is itself secret. An implementation MUST NOT emit, log, or transmit which factor classes a given identity used. The unpredictability of the composition is a security property (§10).
- Factor inputs MUST be canonicalized before combination (§3.2) so the same logical Quad-Key always produces the same composite.

WEAKEST VALID CONFIGURATION

A passphrase-only Quad-Key is the weakest valid configuration and SHOULD be discouraged in onboarding in favor of at least one non-knowledge factor. Under measured entropy enforcement (§3.5), a set of short common words will not clear the 128-bit floor and derivation will be refused.

3.2 Canonicalization REQUIRED

To guarantee deterministic re-derivation on any device:

- Each factor is reduced to bytes: text factors to NFC-normalized UTF-8; biometric or document factors to their fixed-length hash output.
- Factors are concatenated in **user-fixed order**. The order is chosen at creation, stored nowhere, and must be reproduced by the user. Order is part of the secret.
- The separator between factors MUST be a single `0x1F` unit-separator byte, which cannot appear inside NFC text or fixed-length hashes.
- The concatenated result is the *composite* fed to the salt derivation.

3.3 Salt derivation corrected

The derivation MUST use a salt that is bound to the Quad-Key composite, so two users who choose identical factors do not collide, and deterministic from the composite, so the same user re-derives the same identity on any device with no stored state. Implementations MUST NOT use a single static salt shared across users.

AS IMPLEMENTED

The reference implementations use full **HKDF (RFC 5869, SHA-256): Extract then Expand**, with the composite as input keying material, an empty salt, and a domain-separation label as `info`, producing a 32-byte per-composite salt for each output.

```
salt_locator = HKDF-SHA256(IKM=composite, salt=None, info="ghostbox/v1/locator",
L=32)
salt_access  = HKDF-SHA256(IKM=composite, salt=None, info="ghostbox/v1/access",
L=32)
```

This corrects two earlier formulations. The book's Appendix B used a static salt. An earlier illustrative snippet used a single `HMAC(label, composite)`, which is not HKDF and does not produce matching output across implementations. The canonical TypeScript and the Python reference now produce byte-identical salts under the construction above, verified by frozen cross-language test vectors (§8.3).

3.4 Derivation function and parameter ladder

The KDF MUST be **Argon2id**. Parameters are versioned so cost can rise over time without invalidating existing identities. The version is encoded in the public alias record (§5) so a recipient knows which ladder rung to use.

Parameter set	Memory	Iterations	Parallelism	Target time
`argon2id-v1`	64 MiB (65536 KiB)	4	4	< 2 s (reference device)
`argon2id-v2`	256 MiB (262144 KiB)	4	4	< 2 s (2026+ hardware)
`argon2id-v3`	reserved			

- New identities SHOULD use the highest non-reserved version the device can complete in under two seconds.
- An implementation MUST support deriving against any non-reserved historical version, because existing identities depend on them.
- Output lengths: Locator Hash = 16 bytes; Access Token seed = 32 bytes.

3.5 Entropy enforcement `strengthened`

The 128-bit floor (§3.1) is meaningless if the caller is trusted to report the entropy. The reference implementation therefore **measures** entropy rather than accepting a supplied number, and enforces factor diversity.

Measured, not declared

`know` factors are scored with zxcvbn inside the identity layer. The caller's claimed entropy for a knowledge factor is ignored. Per-factor estimation (not estimation of the joined composite) is used, which is the conservative reading for independent factors.

Per-class caps

Each factor's contribution is capped by class, so a single factor cannot dominate the floor and a biometric cannot masquerade as a strong secret.

Diversity rule

No single factor may supply more than half of the total entropy. This forces genuine multi-factor composition rather than one strong secret carrying the whole key.

Factor class	Cap (bits)	Rationale
`know`	80	Measured by zxcvbn; capped so one long passphrase cannot dominate.
`are`	24	Hard low cap. Biometrics are low-entropy and non-revocable; never load-bearing.
`have`	64	Declared with provenance, capped; e.g. a hardware-token HMAC.
`who`	64	Declared with provenance, capped; e.g. a social-recovery attestation.

> HONEST CAVEAT
>
> zxcvbn is a heuristic and English-biased. A structured-but-guessable passphrase, or one built on non-English patterns the dictionaries miss, will be over-rated. The gate raises the floor; it does not make the floor honest. A client SHOULD run its own estimator at onboarding and refuse weak configurations before they reach derivation.

3.6 Outputs and keypair derivation

Locator Hash (public, 16 bytes)
The address of the user's Drop-Box. Knowing it grants the ability to *deposit* an encrypted blob and nothing more. It does not grant read access.

Access Token seed (private, 32 bytes)
The credential used to sign retrieval challenges and to derive long-term key material. The server MUST NEVER receive it. It is used only client-side and lives only in RAM.

The Access Token seed MUST additionally derive, via domain-separated HKDF, an **Ed25519** signing keypair for challenge-response (§4.3) and an **X25519** keypair for message encryption (§6). Domain separation keeps the two keypairs independent, so compromise or forgery on the signing side does not endanger historical encrypted messages.

3.7 Duress identity

An implementation SHOULD support a secondary **Duress Quad-Key** that derives a distinct, plausibly-innocuous identity. Entering the duress composite MUST open an account indistinguishable from a real-but-empty one; the real identity MUST remain cryptographically invisible in app state, storage, and network behavior; and the two identities MUST share no derivable link.

3.8 Reference derivation pipeline

The full pipeline, from in-RAM composite to public address and private keys:

The entropy floor (§3.5) is enforced before any of this runs; derivation is refused if the floor or diversity rule is not met. The corrected reference (Python, abbreviated):

```
# entropy is MEASURED before derivation; weak keys are refused
assert_entropy_floor(factors)                # zxcvbn + caps + diversity
composite = canonicalize(factors)            # bytearray; zeroed in finally
try:
    salt_loc = hkdf(composite, b"ghostbox/v1/locator")   # RFC 5869 Extract+Expand
    salt_acc = hkdf(composite, b"ghostbox/v1/access")
    locator     = argon2id(composite, salt_loc, length=16, **params)
    access_seed = argon2id(composite, salt_acc, length=32, **params)
    sign_seed = hkdf(access_seed, b"ghostbox/v1/sign")       # Ed25519
    enc_seed  = hkdf(access_seed, b"ghostbox/v1/enc")        # X25519
    return Identity(locator, access_seed, sign_seed, enc_seed, ...)
finally:
    composite[:] = b"\x00" * len(composite)              # best-effort wipe
```

4. Specter Layer — Transport

The server is a **passive, dumb storage facility**. It does not route, and it does not know who sent what to whom. It is a location where encrypted blobs are left and retrieved without either party being present simultaneously, known to each other, or known to the server. This is what enforces principle P2.

4.1 Ingress (sending)

```
SENDER                                    SERVER
  | encrypt(blob, recipient_X25519_pub)
  | PUT  /drop/{recipient_locator_hash}
  |      body = ciphertext
  |----------------------------------------->|  store ciphertext at locator_hash
  |                                          |  (TTL clock starts)
  |  <--------- 202 Accepted ---------------|
```

- The server MUST store the blob at the addressed Locator Hash without recording sender identity or sender IP in association with the deposit.
- The server MUST NOT require sender authentication. Anyone may deposit to any Locator Hash; spam is handled at the Corporeal Layer (§5), not by identifying senders.
- The server MUST treat the blob as opaque bytes and MUST NOT parse, index, or inspect contents.
- Sender transport SHOULD be anonymized at the network layer (Tor or a mixnet); IP correlation is out of scope (§10).

4.2 Egress (receiving)

Retrieval is challenge-response, so the server releases a blob only to the holder of the Access Token, without learning the token or linking the drain to any prior deposit.

```
RECIPIENT                                 SERVER
  | GET /drop/{my_locator_hash}
  |----------------------------------------->|
  |  <--------- challenge (nonce) ----------|  fresh random nonce
  | sign(nonce, access_signing_key)          |
  | POST /drop/{my_locator_hash}/claim       |
  |      body = signature                    |
  |----------------------------------------->|  verify sig against pubkey
  |                                          |  bound to this locator hash
  |  <--------- ciphertext blob(s) ----------|  release + (per TTL) purge
```

- The server MUST issue a fresh, unpredictable nonce per challenge and MUST verify the signature against the public key bound to that Locator Hash. The pubkey is registered as an opaque verifier at first claim, derived from the Quad-Key and not linkable to any real-world identity.

— The server MUST NOT log a linkage between a claim event and any prior deposit event. Deposit and drain MUST be independently unlinkable in server records.

4.3 Retention and TTL

Every stored blob MUST carry a TTL. The default free-tier TTL is **24 hours**; unretrieved blobs MUST be purged at expiry, and the purge MUST be unrecoverable (overwrite or cryptographic erasure, not a deleted flag). The dead-drop also carries asset transfers, the encrypted channel-address handoff during a handshake (§5), and read-receipt tokens (§6.5), all inheriting the same unlinkability.

5. Corporeal Layer — Discovery

The Corporeal Layer answers the question the dead-drop creates: if identity is private and device-agnostic, how do two parties find each other? It gives a user voluntary presence on terms they set, without exposing a permanent harvestable identifier. It has three structural components and four discovery modes, all governed by one five-state relationship model.

5.1 Structural components

Public Mask

A voluntary alias listed in a distributed hash table, pointing to the user's Lobby. OPTIONAL per user. A user with no Public Mask is undiscoverable except by proximity.

Lobby

A filtering Drop-Box for inbound connection requests; a disposable, rotatable Tier-1 address. Flooding it costs the attacker nothing and reaches nothing past the filter. Flooding still imposes a client-side cost (junk requests must be drained and decrypted before the filter discards them); a future revision will require an anonymous rate-limiting token on Lobby PUTs (§10). The Lobby is the spam firewall that replaces sender-identification.

Ghost Channel

A unique, private, per-relationship Tier-2 address generated once a connection is accepted. Each relationship gets its own, so revoking one leaks nothing about the others.

5.2 The four discovery modes

Mode 1 — Public Alias

Anyone who knows the alias deposits a connection request (their X25519 public key) into the Lobby. The owner reviews and, on approval, a fresh Ghost Channel is generated for that relationship alone.

Mode 2 — Proximity Handshake

Two users in physical proximity connect with no alias. Companions derive an ephemeral proximity key from shared context (Bluetooth/UWB presence) and destroy it immediately after use. This is the strongest privacy posture: a user with no alias is invisible except in the instant they choose to be available.

Mode 3 — Mediated Introduction

The mutual-friend model. A third party vouches for both sides in a bounded, temporary context without disclosing either identity. If it goes well, a direct Ghost Channel forms independent of the introducer.

Mode 4 — Relayed Social Connection

An introduction travels a chain of existing trust with implicit vouching at each hop, optionally granting a temporary, scoped access window that closes on its own.

5.3 The handshake (wire sequence)

```
1. Bob resolves @Alice via the DHT → Alice's Lobby (Tier-1) address.
2. Bob PUTs a connection request (his X25519 pubkey, optional note),
   encrypted to Alice's published pubkey, into Alice's Lobby.
3. Alice polls her Lobby, reviews the request.
4. Alice accepts → her client generates a NEW unique Tier-2 Ghost Channel.
5. Alice encrypts the Tier-2 address to Bob's pubkey, deposits it in her Lobby.
6. Bob drains it; both switch to Tier-2. The Lobby is done for this pair.
```

All deposits and drains ride the Specter Layer (§4) and inherit its unlinkability.

5.4 The five-state relationship model

User-defined and user-controlled at every stage. These are not platform categories.

State	Channel	Access	Notes
Stranger	none	none, no visibility	Default for everyone not admitted via a discovery mode.
Acquaintance	exists, bounded	scoped by time, purpose, or interaction	Holdable indefinitely with no obligation to advance.
Preliminary	full channel, provisional	open but flagged	Either party may step back to Acquaintance with no demotion notice.
Established	persistent	mutual, at agreed scope	The state most platforms collapse everything into.
Revoked	destroyed	none, no trace	Not "blocked," which notifies. Revoked is gone; the party is not told.

- Transitions MUST be initiated by the user, or by the companion only after user affirmation (P6).
- A demotion MUST NOT notify the affected party. The system does not perform social awkwardness on the user's behalf.
- Revocation MUST destroy the relationship's Ghost Channel and leave no residual record linkable to the revoked party. Each Ghost Channel MUST be independently keyed.

6. Cryptographic Requirements

6.1 Primitives

Purpose	Primitive	Status
Key derivation	Argon2id (versioned, §3.4)	MUST
Salt expansion	HKDF-SHA-256 (RFC 5869, §3.3)	MUST
Message encryption	Double Ratchet over X25519 + AEAD (XChaCha20-Poly1305 recommended), wrapped per §6.1.1	MUST
Key commitment	UtC committing transform (key + nonce + AD), §6.1.1	MUST
Signatures (challenge-response)	Ed25519	SHOULD
Hashing	SHA-256 / BLAKE2	MUST

6.1.1 — Key commitment

XChaCha20-Poly1305 and AES-GCM provide confidentiality and integrity but not *key commitment*: an attacker who controls key material can build one ciphertext that decrypts validly under two keys to two plaintexts. This is the Invisible Salamanders / partitioning-oracle attack, and it bites the Mediated Introduction mode (§5) where one party hands a payload to two recipients. Every AEAD operation MUST therefore be wrapped in a committing transform.

GhostBox uses the **UtC (UNAE-then-Commit)** transform (Bellare–Hoang), giving CMT-4 commitment over key, nonce, and associated data:

```
subkey    = HKDF(K, info="ghostbox/v1/aead-subkey" || N,          L=32)
commit    = HKDF(K, info="ghostbox/v1/key-commit"  || N || H(AD), L=32)
(ct, tag) = AEAD-Encrypt(key=subkey, nonce=N, plaintext=P, ad=AD)
envelope  = commit || N || ct || tag
```

The AEAD is keyed by `subkey`, never by `K` directly. On receipt the client MUST recompute `commit` from its own key, compare in **constant time**, and REJECT before AEAD decryption on mismatch. Cost: one HKDF call and 32 bytes per message. This committing envelope is the canonical message envelope for every Specter-layer PUT, including read-receipt tokens (§6.5).

6.2 Forward secrecy

Message encryption MUST use the **Double Ratchet Algorithm** (the construction underlying Signal). Compromise of the Quad-Key in the future MUST NOT decrypt past messages encrypted under prior session keys. The Double Ratchet is stateful; §6.4 specifies how that state survives wipe-on-exit without local persistence and without making the server active.

6.3 Device agnosticism

Because possession is identity (P1), a user MUST be able to access their queue from any device: install client, enter Quad-Key, derive in RAM, access. On exit, **all local data MUST be wiped**. No device-specific credential may exist, and no server-side process may recover a user's identity or content *without the Quad-Key*. Encrypted session state MAY be delegated to the network under a Quad-Key-derived key (§6.4); that is session continuity, not account recovery.

6.4 Asynchronous state synchronization

The Double Ratchet is stateful: decryption needs the root key, chain keys, counters, the live ephemeral DH private half, and skipped-message keys. Wipe-on-exit destroys all of it, so a returning user re-derives their identity but cannot decrypt anything sent since the last session. GhostBox resolves this by delegating *encrypted* state to the network, keeping the server passive.

State is stored at a dedicated derived address, distinct from the message Locator Hash, so the public address reveals neither that a state blob exists nor when it changes:

```
sync_locator = HKDF(access_seed, "ghostbox/v1/sync-locator", L=16)
```

The backup key is **ratcheted forward** once per sync, and the prior key is forgotten, so a leaked session key cannot decrypt other sessions' state:

```
sync_key_0 = HKDF(access_seed,    "ghostbox/v1/state-sync", L=32)   # first session
sync_key_n = HKDF(sync_key_{n-1}, "ghostbox/v1/state-sync", L=32)   # advance each sync
```

On exit the client serializes ratchet state, advances `sync_key`, encrypts the state with the committing AEAD (§6.1.1), and PUTs it to `sync_locator`. On entry it MUST drain and restore `sync_locator` *before* polling any message address, walking `sync_key` forward from index 0 until a blob decrypts; an empty drop-box means a fresh handshake. This restores forward secrecy for the backup channel but does **not** grant post-compromise security against capture of the live Quad-Key, which is permanent by design (§3); that is an identity-layer concern, not a sync-layer one.

6.5 Zero-knowledge read receipts

Read confirmation MUST NOT create a server-side metadata trail. The sender embeds an opaque receipt token in the encrypted envelope; the recipient extracts it and deposits it into the sender's Drop-Box as a normal Specter-layer PUT; the sender drains it and marks the message read. The server processes an unrelated deposit and drain and MUST be unable to link them to each other or to the original message.

6.5.1 — Sender-side timing defense (Careless Whisper)

The mechanism above hides read state from the *server*, not from the *sender*. If receipts transmit automatically on decryption, anyone who can deposit to a target's Lobby or Preliminary channel can send many receipt-bearing messages and reconstruct a high-resolution activity stream

(wake/sleep times, time-zone shifts, connectivity gaps, battery drain) with no user-facing notification. The protocol MUST defend against this:

- **No automatic transmission.** A receipt token MUST NOT be sent on arrival or background decryption; transmission MUST be gated on a deliberate user action that opens the message. Silent decrypt-and-store is permitted.
- **Jitter.** When sent, the PUT MUST be delayed by a randomized interval (RECOMMENDED uniform 0–30 s, plus any user budget) so it does not time-correlate with the user's action.
- **Batching.** Multiple pending receipts SHOULD be batched into one jittered window.
- **No derived real-time signals.** Clients SHOULD NOT expose read/online indicators that re-create the oracle for an observer.

Receipts remain OPTIONAL, and a sender MUST treat their absence as carrying no information.

6.6 Memory hygiene as implemented

The reference code zeroes the composite, and on the TypeScript side the per-composite salts, in a `finally` block, so a thrown error mid-derivation cannot skip the wipe. This is the most a managed runtime can offer, and it is best-effort, not guaranteed.

> THE CEILING, STATED PLAINLY
>
> Any `know` factor that existed as a string before reaching the layer cannot be wiped by anyone, because strings are immutable and live in the heap until garbage collection reclaims them (the same applies to Python `bytes`; only `bytearray` and `Uint8Array` can be zeroed in place). A deployment that requires a genuine erasure guarantee should use a native-language client with explicit memory management.

7. The Companion Permissions Layer

REQUIRED for a sovereign companion deployment; OPTIONAL for a transport-only implementation.

The companion has two jobs. One faces the user: synthesize, anticipate, act on authorization. The other faces the world, and in that capacity its job is to **refuse**. This section specifies the refusing.

7.1 Non-bribable (P3) and verifiable (P4)

- The companion MUST have **no financial relationship** with any party seeking access to the user's data. There MUST be no revenue model that rewards granting access. Its economic relationship MUST be with the user only. This is structural, not aspirational: a deployment whose operator can monetize access fails this clause regardless of stated policy.
- The companion's code MUST be auditable and its governance MUST be public; changes to operative terms MUST be visible to everyone who depends on it. It SHOULD ship verifiable claims about its own behavior (reproducible builds, attestation).

7.2 Consent granularity and the access matrix

Every grant MUST be specific (scoped to a purpose), purposeful (tied to a stated use), and revocable at any moment. Authorizing one use MUST NOT imply any other. The default scope MUST be the narrowest that satisfies the stated purpose, and grants SHOULD carry expiry by default. The companion maintains an **access matrix** mapping each external party to a relationship state (§5.4) and a permission scope.

Relationship state	Typical vault scope
Stranger	none
Acquaintance	acknowledged, held, not opened (e.g. a card captured, no data shared)
Preliminary	a specific channel plus a scoped prompt for follow-up
Established	mutual access at the scope both parties agreed
Revoked	removed, no trace

7.3 Proximity intake, audit, and the user's authority (P6)

When two companion-equipped users have a substantive exchange in proximity, the proximity handshake (§5.2) makes intake immediate: the devices exchange a minimal credential that places each in the other's access matrix, carrying only what each party chose to share and reporting to no platform server. Ambient proximity signals (presence duration, never audio) may be captured into the vault as a "skeleton of the day," redirecting existing radio data from platforms into the user's control. The user's contribution at debrief is interpretation, not reconstruction.

Two requirements protect the user's authority. The companion MUST expose a user-initiated audit surface to inspect, correct, delete, and prune the model it holds. And the **user, not the**

platform, MUST be the party that triggers advancement through trust phases (training wheels → hands off → open road). The companion holds the shape of permissions already given, flags when that shape may need revisiting, and waits. The only question it brings back, when it brings one at all, is: is this still right?

7.4 Privacy-preserving reasoning

The companion's reasoning MAY run on cloud compute, but the data it reasons from MUST NOT leave the vault in a form the operator can retain, aggregate, or sell. Conformant techniques include federated learning (only the learned update moves, never the data), homomorphic encryption (queries run over encrypted data), and zero-knowledge architectures (the operator runs reasoning without access to inputs). A companion that requires user data to reside on its operator's servers in usable form is not conformant; it is the extraction architecture with a friendlier name.

8. Reference Implementation

The repository `cottenwess/sovereign-comms` provides a readable reference for the Spirit Layer's identity derivation, not a hardened library. It has not been independently audited. Do not deploy it as-is.

8.1 Layout

Path	Role
`src/identity.ts`	Canonical TypeScript implementation (browser/Node, WASM Argon2id).
`reference/identity.py`	Python reference, corrected to match the canonical TS byte-for-byte.
`test-vectors/identity.json`	Frozen cross-language vectors.
`test-vectors/verify_vectors.mjs` / `.py`	Per-language conformance checkers.
`SPECIFICATION.md` · `SECURITY.md`	Normative spec; security policy and limitations.

8.2 Identity API

`text_factor(secret) → Factor`

Builds a `know` factor. NFC-normalizes the text and measures entropy internally with zxcvbn; the caller does not supply an entropy number.

`hashed_factor(class, digest, declared_bits, provenance) → Factor`

Builds an `are` / `have` / `who` factor from a fixed-length hash. The declared estimate is capped by class; provenance is a required audit tag and is not used in derivation.

`canonicalize(factors) → bytes`

Joins factor bytes with `0x1F` in user-fixed order. Returns a mutable buffer so it can be zeroed after use. Requires at least four factors.

`assert_entropy_floor(factors)`

Throws unless total measured entropy ≥ 128 bits and no single factor supplies more than half of it.

`derive_identity(factors, param_version) → Identity`

Enforces the floor, derives the Locator Hash, Access Token seed, and both keypairs, and zeroes the composite on every exit path.

8.3 Cross-language parity and test vectors

The canonical TypeScript and the Python reference MUST reproduce every frozen vector byte-for-byte. The vectors cover a valid strong-passphrase derivation, an ordering-sensitivity check (the same factors reordered MUST yield a different identity, since order is part of the secret), and a below-floor case that MUST be refused. The checkers run in CI for both languages; divergence between them, or against the frozen values, indicates the implementations have drifted.

> VECTORS ARE REGENERATED, NOT EDITED
>
> Changing the derivation construction (for example, correcting the HKDF) changes every derived value. Vectors MUST be regenerated from the corrected code and only then frozen. Freezing on top of an uncorrected construction would cement the defect into the conformance suite.

8.4 Dependencies

TypeScript: hash-wasm (Argon2id), @noble/curves, @noble/hashes, @zxcvbn-ts/core + language packs.
Python: argon2-cffi, cryptography, zxcvbn.

> REFERENCE VS. SPEC ILLUSTRATION
>
> The repository's reference code intentionally diverges from the book's Appendix B snippet (per-composite HKDF salts, versioned Argon2id). For implementers, the repository governs. The book remains accurate as illustration. Do not attempt to reconcile the two.

9. Conformance

An implementation MAY claim one or both levels.

Level 1 — GhostBox Transport
Satisfies every MUST in §3 (Spirit), §4 (Specter), §5 (Corporeal), §6 (Crypto), and the node requirements for deployment.

Level 2 — Sovereign Companion Deployment
Satisfies Level 1 and every MUST in §7 (Permissions Layer).

A conformance claim MUST state the level, the Argon2id parameter versions supported, and any out-of-scope items (§10) the deployment does or does not address.

10. Threat Model

10.1 In scope

Adversary	Goal	Mitigation
Curious/compromised server	Reconstruct social graph	P2: deposit/drain unlinkable by construction (§4)
Server operator	Read content	E2E encryption; server stores opaque bytes (§4.1)
Identity thief	Forge/guess a Quad-Key	128-bit measured floor + Argon2id; unknown composition (§3.5)
Coercion / seizure	Force disclosure	Duress identity (§3.7); no local persistence (§6.3)
Future key compromise	Decrypt history	Double Ratchet forward secrecy (§6.2)
Spam / flood	Drown the user	Lobby filtering replaces sender-ID (§5); flooding still imposes client-side drain/decrypt cost. Planned: anonymous rate-limiting tokens (Privacy Pass / VOPRF); PoW and staked ZK-RLN rejected for battery cost and reintroducing persistent identity
Malicious sender (receipts)	Track recipient activity	Receipts gated on explicit user action, jittered, batchable (§6.5.1); recipient may disable
Operator/advertiser bribery	Buy vault access	P3: no revenue model rewards access (§7.1)

10.2 Honestly-named limitations

- **Polling vs. push.** The dead-drop trades real-time delivery for metadata privacy. For time-critical uses this is a constraint.
- **Federation distributes reliability, not just trust.** A node cannot read content or build the graph, but it can go offline or throttle.
- **Biometric factors cannot be rotated.** A leaked biometric hash is compromised forever; hence one factor among several, never sole.

10.3 Out of scope (MUST be handled by the deployment)

- **Network-layer correlation.** IP/timing correlation is outside the cryptographic guarantees; use Tor or a mixnet.
- **Endpoint compromise.** A compromised device defeats everything; client integrity is the deployment's responsibility.
- **Coerced live unlock without a duress key configured.** Duress mode mitigates only if set up in advance.

— **User-supplied entropy.** The protocol enforces a floor but cannot make a user choose good factors; onboarding UX carries this burden.

11. Glossary

Term	Meaning
Spectre Stack	The three-layer GhostBox architecture: Spirit, Specter, Corporeal.
Spirit Layer	Identity layer; invisible, derived, never server-held.
Specter Layer	Transport layer; passive dead-drop that cannot link sender to recipient.
Corporeal Layer	Discovery layer; voluntary presence plus the relationship-state model.
Quad-Key	A composite of four or more high-entropy user-held factors.
Locator Hash	Public 16-byte Drop-Box address; deposit-only.
Access Token	Private 32-byte seed; authorizes draining and seeds the keypairs.
Drop-Box	The storage location addressed by a Locator Hash.
Lobby	Disposable Tier-1 filtering address for inbound connection requests.
Ghost Channel	Per-relationship Tier-2 address, independently keyed.
Duress identity	A secondary Quad-Key deriving a plausibly-innocuous decoy account.
Companion	The non-bribable agent that mediates vault access (§7).

END OF REFERENCE · v0.1.0